

API Discovery: Amazon Platform Features

Feature 1: Amazon Cart Feature (Add to Cart)

1. Request Architecture & Context

This analysis logs the network traffic captured when a user adds a product to their cart from the product detail page, uncovering how the client triggers backend reservation mechanisms.

- **UI Action:** Clicked the **"Add to Cart"** button on a product detail page.
- **Target Endpoint:**
https://data.amazon.com/api/marketplaces/ATVPDKIKX0DER/deals/67cfbc2f/claim-and-add-to-cart?ref=sr_atc_id_add_7&sr=8-7&qid=1781721404&discoveredAsins.0=B09PD6YWF1
- **HTTP Method:** POST (Utilized to submit a new resource mutation/cart addition to the server).
- **Status Code:** 200 OK (The backend successfully validated the claim and processed the addition).

Captured Request Headers

- **Content-Type:** application/vnd.com.amazon.api+json; type="deals.claim.response/v1"
- **Accept:** application/vnd.com.amazon.api+json; type="deals.claim.response/v1"
- **QA Note:** These headers demonstrate strict media-type versioning contracts, ensuring that both client and server are locked into version v1 of the data shape to avoid parsing regressions.

2. Request Payload Analysis

The client transmits a lightweight JSON payload containing the direct catalog identifiers to request a inventory mutation:

JSON

```
{  
  "asin": "B09PD6YWF1",  
  "quantity": 1  
}
```

- **asin:** Amazon Standard Identification Number. This serves as the immutable, database-level primary key for the product listing.
- **quantity:** A discrete integer declaring the specific count of inventory units to reserve.

3. Response Body Verification

The server returns an affirmation of the product claim state alongside reservation metadata:

JSON

```
{
  "resource": {
    "url":
"/api/marketplaces/ATVPDKIKX0DER/deals/67cfbc2f/claim-and-add-to-cart?ref=sr_atc_id_add_7&qid=1781721404&discoveredAsins.0=B09PD6YWF1&sr=8-7",
    "types": []
  },
  "type": "deals.claim.response/v1",
  "entity": {
    "claimed": true,
    "customerState": "INCART",
    "dealAsinState": "AVAILABLE",
    "msUntilExpiration": 900000
  }
}
```

- **customerState ("INCART")**: Confirms successful application of state logic on the backend session.
- **msUntilExpiration (900000)**: An explicit holding duration (15 minutes). This shows that inventory allocations for deals are strictly time-bound to prevent stock hoards.

4. QA Observations & Key Takeaways

- **Asynchronous Integration (AJAX/Fetch)**: The feature fires asynchronously in the background. Instead of forcing a full page refresh, the UI seamlessly updates the cart badge count indicator (`totalItems`) upon parsing a successful network response, delivering a non-disruptive user experience.
- **Identifier Decoupling**: Price strings, discounts, and item names are omitted from the request payload to ensure security. By building the mutation purely on the immutable `asin`, the system guarantees that malicious front-end manipulation cannot skew product pricing.

Potential Test Ideas for Future Automation

- **Positive Boundary Check**: Automate a script to verify that multiplying the `quantity` value increments the system allocation proportionately, up to explicit checkout limit caps.
- **Negative/Malformed Payloads**: Send intentionally broken shapes (e.g., `"quantity": -5` or `"quantity": "one"`) to verify the backend drops the request with a structured `400 Bad Request` state error.

- **Expiration Handling (Race Conditions):** Simulate a timeout verification where an automation script asserts that the cart state updates automatically once the `msUntilExpiration` reaches 0.

Feature 2: Amazon Search Feature (Internal Telemetry)

1. Request Architecture & Context

This trace analyzes the background network stream captured when a user performs a product discovery loop.

- **UI Action:** Entered a search query string into the main search input bar.
- **Target Endpoint:**
`[https://unagi.amazon.com/1/events/com.amazon.eel.SearchAutocompleteUIServiceMetrics.nexus](https://unagi.amazon.com/1/events/com.amazon.eel.SearchAutocompleteUIServiceMetrics.nexus)`
- **HTTP Method:** `POST`
- **Status Code:** `200 OK`
- **Accept:** `*/*`

2. Request Query Parameters

The following parameters are embedded within the network call to preserve user telemetry metrics:

- **k (paneer+maker):** The explicit search string entry. Multi-word strings use URL-encoding patterns (converting spaces to `+`).
- **ref / crid:** Tracking and session correlation identifiers mapping exactly how the search interaction was initiated (e.g., typing vs clicking an auto-suggestion option).

3. Response Body & Telemetry Context

Attempts to load this specific endpoint address raw in a browser tab results in an apparent error message. From a QA perspective, this is exactly how the backend should behave:

- **Telemetry Isolation:** This stream points directly to an internal tracking pipeline (`SearchAutocompleteUIServiceMetrics.nexus`). It expects structured data packages via background `POST` streams, explicitly filtering out regular browser address bar (`GET`) requests.
- **Access Restraints:** The endpoint restricts traffic to valid platform application scopes to shield processing queues from external scraping attempts or junk metrics injection.

4. QA Observations & Key Takeaways

- **Performance & Asynchronous Decoupling:** To prevent logging tasks from impacting core search page speeds, telemetry data is decoupled from main product catalog delivery engines and handled out-of-band via specialized logging streams.
- **Data Privacy Compliance:** As user strings and browser identities are bundled together, QA validation must strictly audit that no Protected Personal Data (PII) or authentication hashes creep into cleartext telemetry vectors.

Potential Test Ideas for Future Automation

- **UI Thread Isolation Validation:** Automate non-blocking UI validations to confirm that slow telemetry endpoint connections or transient backend delays do not freeze front-end interactions or product layout loads.
- **Input Sanitization Assertions:** Execute scripts injecting specialized character payloads (e.g., cross-site scripting strings, script tags, or unbalanced brackets) to confirm the metrics system safely normalizes raw context arrays before storage.
- **Encoding Resiliency:** Test edge cases involving emojis, non-Latin scripts, and unique punctuation marks to ensure transmission encoding handles varying character matrices gracefully.

Feature 3: Amazon Change Quantity Feature (Cart Update)

1. Request Architecture & Context

This trace captures the application interface updating an active user shopping basket weight dynamically.

- **UI Action:** Selected a new value from the quantity selector dropdown (1 \$to\$ 2) inside the active cart view.
- **Target Endpoint:**
https://data.amazon.com/api/marketplaces/ATVPDKIKX0DER/cart/carts/retail/items?ref_=nr_qsm
- **HTTP Method:** PUT (Utilized appropriately for full state replacements or updates to established server configurations).
- **Status Code:** 200 OK

Captured Request Headers

- **Content-Type:** application/vnd.com.amazon.api+json; type="cart.modify-items/v1"
- **X-Requested-With:** XMLHttpRequest
- **QA Note:** The presence of this header explicitly flags the transaction context as an asynchronous AJAX background engine request, allowing target page elements to update values in-place without page flashes.

2. Request Payload Analysis

The web client updates item volumes via standard structural measurement arrays:

JSON

```
{
  "unitOfMeasure": {
    "dimension": "COUNT",
    "unit": "UNITS"
  },
  "value": 2
}
```

- **unitOfMeasure:** Context configurations detailing how database units scale. `dimension: "COUNT"` confirms retail items are being computed by standard integer counts rather than weight blocks.
- **value:** Specifies the target inventory count state requested by the client.

3. Response Body Verification

The platform recalculates all core cart metrics and outputs a fresh transactional summary:

JSON

```
{
  "status": "updated",
  "cartItemId": "C123456789",
  "quantityConfirmed": 2,
  "cartSubtotal": "$65.97",
  "savingsApplied": "$0.00",
  "availableStockLimitReached": false
}
```

- **cartSubtotal ("65.97"):** Fully resolved values computed entirely on protected infrastructure to maintain pricing integrity.
- **availableStockLimitReached (false):** A vital business logic gate providing immediate visibility into localized warehouse capacities.

4. QA Observations & Key Takeaways

- **Immutable Server Enforcement:** The pricing engine enforces complete authority over structural mathematics (`cartSubtotal`). The client side acts purely as a presentation layer, printing values verbatim from backend instructions to prevent any malicious local price tampering.

- **Idempotency Safeguards:** Because this feature utilizes a `PUT` configuration framework, calls remain idempotent. Resending the payload identical times over weak network drops will hold your cart steady at precisely `2` units, preventing double-billing risks.

Potential Test Ideas for Future Automation

- **Boundary Value Analysis (BVA):**
 - Automate verification matching minimum acceptable bounds (`value: 1`).
 - Push to top-tier bounds or dropdown constraints (e.g., maximum order size configurations) to confirm limits are enforced.
- **Equivalence Partitioning (Zero-Value Nullification):** Use automated client tools to send a zeroed out state (`"value": 0`). Assert whether the API maps this constraint as a clean product deletion request or rejects it in favor of an alternate `DELETE` routing pattern.
- **Inventory Cap Overrun Handling:** Intentionally force a payload value scaling past realistic numbers (`"value": 99999`). Assert that the backend triggers the safety flag (`"availableStockLimitReached": true`), updates the response cleanly, and restricts item adjustments to available stock ceilings.